

Project שטמח: Schmate* (Pronounced “SHMAH-teh)

Extending re-Isearch with vector datatypes for embeddings. Enabling a better dense passage retrieval (DPR) for retrieval augmented generation (RAG) and hybrid search.

(*) Schmate means RAG in Yiddish

Edward C. Zimmermann // re-Issearch.org // 20 November 2025

The Goals: Be **Powerful**, **Flexible** and **Efficient**

Its design focus is for consumer grade hardware and edge computational devices (including a number of SOCs such as Qualcomm Snapdragon and NVIDIA Jetson platforms).

“It's one thing to have a good search but it must also be economically feasible, efficient and sustainable.”

Key needs:

- To create a highly flexible, computationally constrained future forward semantic search technology building block that can be integrated within re-lsearch (for Hybrid Search) but also used without it.
- 100% Open Source. Apache 2.0 License
- A model to support additional algorithms for Aproximate Nearest Neighbour (ANN) search as use-cases arise.

==>

- We elected for “easy” integration of “current” and future models to support llama.cpp (and bert.cpp): gguf and ggml.
- For now we start off with a turbo-charged and significantly enhanced implementation of HNSW (forking the popular HNSWlib C++ code)

Hybrid Search: At its core is Re-Isearch

Re-Isearch is a 100% open source (Apache 2.0) novel multimodal search and retrieval engine using mathematical models and algorithms different from the all-too-common inverted index. It is a kind of hybrid between full-text, XML, object and graph noSQL-db that natively ingests a wide range of document types and formats. It has been open-sourced through a grant from [Nlnet/NGI-Zero Search](#).

See our talk from FOSDEM '22: [A lightning intro to re-Isearch](#).

https://archive.fosdem.org/2022/schedule/event/lt_re_isearch/

(Re-)lsearch History

The past 30 years

- lsearch was a legendary open-source text retrieval software first developed in 1994 as part of the lsite Z39.50 information framework with support from NSF.
- Development was divided between CNDIR/MCNC (North Carolina, USA) and Bsn (Munich, Germany).
- In 1998 BSn launched a proprietary fork called IB using new algorithms that significantly improved performance and power.
- IB development shifted to BSn's research arm: NONMONOTONIC Lab. It was deployed in 100s of high profile sites through BSn projects as well as partners.
- In 2011 Bsn/NONMONOTONIC's proprietary fork ceased with Ed Zimmermann leaving his active role in BSn and joining CIB.
- The software moved to the proverbial attic.
- Even ten years on from End-of-Development and despite lack of support some servers were still running!???
Don't break a working system.....

Who used it? Among others...

The U.S. Patent and Trademark Office (USPTO) patent search, the Federal Geographic Data Clearinghouse (FGDC), the NASA Global Change Master Directory, the NASA EOS Guide System, the NASA Catalog Interoperability Project, the astronomical pre-print service based at the Space Telescope Science Institute, The PCT Electronic Gazette at the World Intellectual Property Organization (WIPO), the SAGE Project of the Special Collections Department at Emory University, Eco Companion Australasia (an environmental geospatial resources catalog), the Open Directory Project, genomic search for the Australian National Genomic Information Service's (ANGIS) human genome project (and its eBiotechnology workbench split-off); the D-A-S-H search portal against racism, antisemitism and exclusion (funded within the framework of the action program "Youth for tolerance and democracy - against right-wing extremism, xenophobia and anti-Semitism", the YOUTH program of the European Community and with additional support from the German Federal Agency for Civic Education); the e-government search (Yeehaw) of the U.S. State of Utah to agronomic cooperation across the Mediterranean region (supported by the EU's DG). Integrated into a number of CMS platforms it powered search for a number of high volume web sites. It was also used as a database accelerator by a number of eCommerce shops.

Re-Isearch. Its algorithms

Reborn in 2020 in the middle of the global Covid19 pandemic as Project re-Isearch catalysed by a grant from NInet/NGI-Zero. Like the original it is not just about textual words but pushes the envelope.

- Based roughly on pat-arrays going back to the original work of Gonnet, Baeza-Yates, Uri Mamber et al.
- The algorithm is accelerated by a simple trick: In contrast to depending just on the addresses it also stores a cache of the first X characters and the address range in the index to significantly save on system calls and I/O. Since this can be mapped into memory it super-scales to multiple processes accessing the same indexes.
- Each word has a composite address made up from the address (key) of the file it is contained in, the file offset to the start of record (a file can contain multiple records) and the offset from the start of record to the start of the word.
- Each field (container) instance encountered gets added in a separate file its start and end address stored. We use multiple files to better exploit individual file system features, configurations and the potential tuning thereof..
- The vector of the pairs <word, address> is sorted by word. The address column is dumped to disk, the addresses whose words match the first and/or last X characters is dumped as < word fragement, start offset, end offset> where are offsets are into the index above.
- The address of every word is stored and the address range of every field instance is stored.
- An index file tracks the correlation of composite address to file and record.
- Should a path/field/node be of an extended datatype, a handler parses the contents and adds it to a datatype optimised index (date, numerical, geospatial etc.).

What did this deliver?

Unlimited term length, unlimited fields, wildcards and deep search

- For every word we have an address and can open the file and read it. The word could have been encoded. No general need for an intermediate file store (such as JSON). How it was encoded we know from index time and can have classes that are responsible for handling the document format do the right magic.
- Since we know the addresses in all our fields (paths) we can find the paths for any word.
- This lets us, for example, not just search for words in a specific field, a specific path but also for words in the same container without knowing its name or path.
- Since we can read the original file we can also search for unlimited length literals using any wildcard or regex scheme of our dreams—even applied to field names and paths.
- We can even go the other way and ask: what words are in a given field or path instance?
- Since the hierarchical structure of a document is modelled we can walk up (parent) and down (children) from any point in its tree without need to re-parse.
- Since we know all the words and the structure we can even at search reconstitute an alternative encoding without needed to re-parse the original. Its just load and translate using the extracted structure and addresses from the indexing stage.
- Throw in multiple polymorphic datatypes with their own search and ranking algorithms...

Exploiting Structure

(Explicit and Implicit)

The re-lsearch engine exploits document structure, both implicit (XML and other markup) and explicit (visual groupings such as paragraph), to zero in on relevant sections of documents, not just links to documents. These are a heterogeneous mix of text, data (a [large number of datatypes](#) including: numerical, computed, range, date, time, geo, boolean etc. as well as a number of hashes including several phonetic), network objects and databases. These datatypes have their own, for their individual datatypes, storage and retrieval algorithms (including relevant ranking and similarity methods).

Project Schmate extends re-lsearch with vector datatypes tuned for embeddings and using structure not just for chunking but for a more optimal passage retrieval.

Why do we need/want “semantic search”
— embeddings (dense vectors)?

Are not “keywords” (lexical search)
sufficient if not often, in the right hands,
superior?

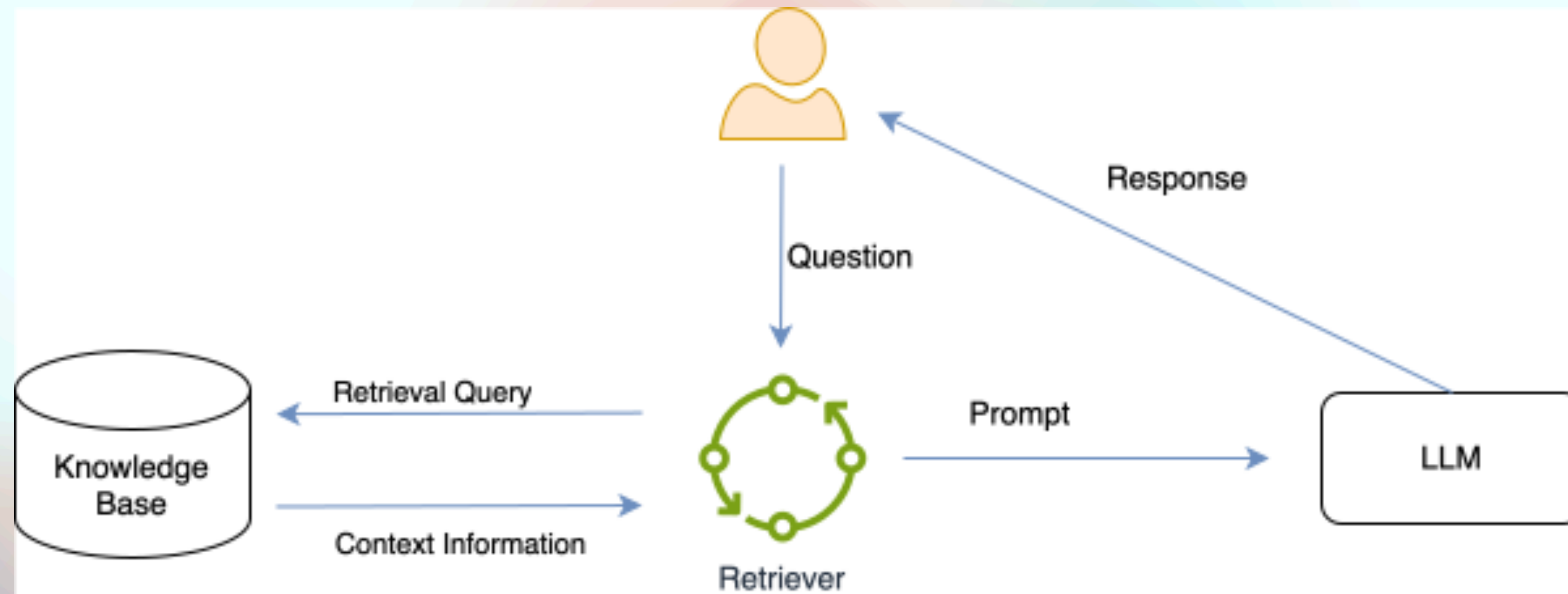
Semantic Search

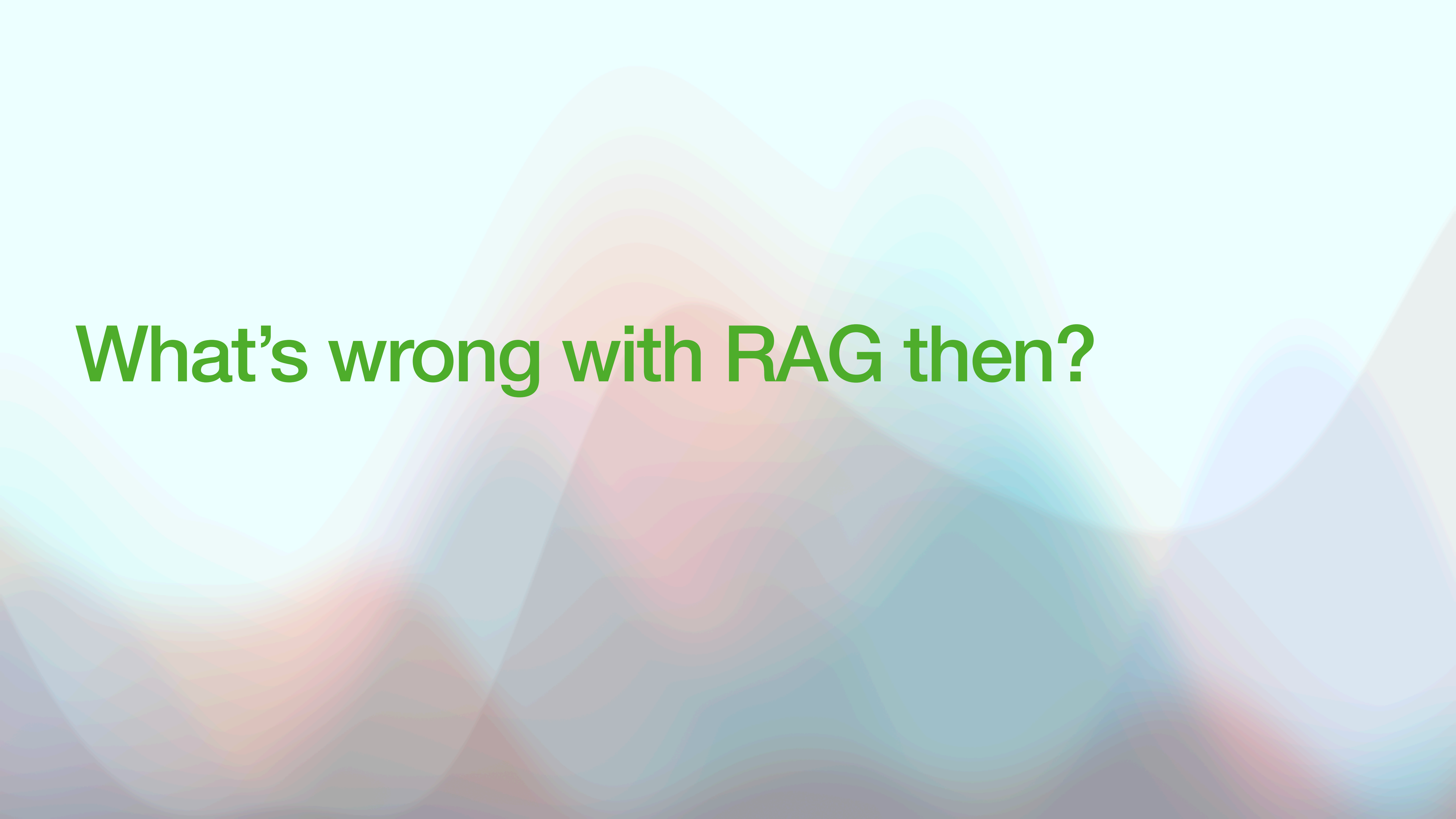
- **Not everyone is a trained library scientist.**
- **Enhanced user experience:** When search results are highly relevant, users have a more satisfying experience. They can quickly find the information they need without wading through pages of irrelevant links.
- **Improved relevance:** By understanding the meaning behind a search query, especially complex or ambiguous ones, search engines can deliver more relevant results. This means users are more likely to find exactly what they're looking for on the first try.
- **Increased engagement:** Relevance is key to engagement. When users find what they are looking for, they are more likely to spend time interacting with the content since they more quickly find what they're looking for.

Large Language Models as Search Ersatz?

- Everyone by now has played around with them, be it Chat-GPT, Claude, Gemini, LLaMA, DeepSeek, ... Many see them as an “ersatz” for traditional search engines. But ...
- Large language models only “know” what they have been trained on. Since the world does not stand they would need to be re-trained. This is computationally extremely expensive.
- LLMs have poor “memory” beyond the session (and there are often limits here too).
- Most of the foundational LLMs we tend to use have been trained by others. They don’t know “my data”.
- The answer is the **R**etrieval **A**ugmented **G**eneration paradigm. RAG fuses indexing technology with LLMs.
- RAG allows developers to provide the latest research, statistics, or news to the generative models. They can use RAG to connect the LLM directly to live social media feeds, news sites, or other frequently-updated information sources. The LLM can then provide the latest information to the users.

RAG uses search to fortify prompts to deliver information out of scope of the original training of the LLM.





What's wrong with RAG then?

Typical RAG challenges or why Re-Issearch+Schmatte for DPR?

While creating a prototype RAG application is these days comparatively easy thanks to sites like LangChain and HuggingFace, making it work well much less performant, robust, or scalable to a large knowledge corpus has proven for many organizations as quite difficult.

1. Ingest

A. Data Extraction

B. Chunk size and chunking strategy

C. Robustness and scalability

2. Search and Retrieval

3. Data Security

Ingest

A) Extraction

Extracting data from diverse types of documents, such as emails, PDFs and office files such as ODF can be challenging. Documents have generally both **explicit** and **implicit** structure (text formats of what is typically called unstructured is not really without structure). The re-Issearch engine already understands these (and more than 80 base types and with these 100s more and with a plugin-in architecture easily extended to support new formats).

Our ODF parser is, for example, the absolute state-of-the-art: the fastest and most accurate (created in coordination with the OASIS ODF TC).

Ingest

B) Chunking

Our approach is variant of context-aware chunking that also takes the hierarchical structure of text into account.

It contains a computationally more modest approach than “agenic” (which uses a LLM to determine chunks) to find better context matches by using the found passages and seeing them in the hierarchical context of the document. In contrast to recursive we view the edge nodes of a document tree as the fine level chunks and determine the optimal passage on the distribution and relation of the retrieved chunks in the tree.

Ingest

C) Robustness and Scalability

In order to be robust and scale one needs a modular and distributed system. The re-lsearch design has been proven over many years in a large number of large scale highly visible deployments.

Beyond the internal document formats (DOCTYPES) there is a mechanism for dynamic loadable plugins.

In the future a plug-in design will also be implemented to extend re-lsearch to support additional ANN (approximate nearest neighbour) algorithms for dense vector retrieval ranking as well as embedding generators.

The 64-bit index (32-bit file IDs) supports a max Input of 16384 Tbytes (Total of all files) and 1 million records per physical index which, in turn, can be aggregated on demand at search-time with up to 254 additional indexes to create a searchable virtual index (max 255 million records). This can be significantly increased by compiling for 64-bit file IDs. In the other direction it can be compiled to produce 32-bit indexes to save on storage.

Search and Retrieval

The power of re-Search as a hybrid

- a. Because of their chunking size and strategy retrieved passages may sometimes been misaligned to context. Because we support multi-indexes, have a scope aware passage retrieval system-- we retrieve not just the chunks in the similarity top-k but view these in their contextual scope to retrieve a scope that is more inclusive of probable context.
- b. We also do, among many other things, query augmentation to help contextualize and generate accurate responses, We can also deploy query routing to the most appropriate domain (index or even virtual collection of indexes).

Sustainability

Energy and Computational resource demands.

It's one thing to have a good search but it must also be economically feasible, efficient and sustainable.

While the standard re-Search powered node can run on pretty much any platform, even small embedded systems, LLM inferencing (currently) demands a bit more resources.

But instead of costly GPU data-servers (where an NVIDIA GB200 can cost up to \$70,000 each in a rack-scale NVL72 system—even the cut-price Asian market 96GB H20 costs \$12k— we aim to target accessible platforms.

While companies such as Google sing praise about their massive context length (large context or LC) models they are heavily data-center based and energy intensive. By constraining computational requirement we can leverage existing hardware and adhere to our aim to eliminate the need for costly data centers and reduce the environmental impact for search.

Just as with the standard version, it is intended that users will be able to host their own data under their own well defined conditions.

Flexibility and Future Forward

Search must be economically feasible, efficient and sustainable.

We have opted for a flexible architectural design to support multiple vector stores including, of course, HNSW. The basic idea is to view them as data types and within their handlers pipelines to selectable/extendable embeddings models.

We have found, for example, that some algorithms work for some data really well but for others sub-optimally and visa-versa. Another aspect of our design, however, is to enable a simple pipeline where documents handed over to the ingest get processed by the appropriate document handler and data types detected, structure exploited for chunks, embeddings created and stored in an appropriate vector data store.

We currently use for embeddings both bert.cpp and llama.cpp as BERT support was added to llama.cpp in February 2024 via PR #5423 [GitHub](#). Unfortunately llama.cpp has since dropped support for ggml.

Schmate v1. Indexes

Semantic search with SBERT + GGML/GGUF + HNSWlib.

This system provides a sentence-embedding search engine built on:

- SBERT (Sentence-BERT) model running via ggml/gguf (no Python dependency)
- HNSWlib for fast approximate nearest neighbor (ANN) retrieval
- Memory-mapped offset files for persistent, efficient text-embedding linkage
- Optional re-scoring for quantised embeddings (such as binary) via an LSM vector store that supports: in-memory and memory mapped access.
- LRU caching of indexes using available RAM to prevent both swapping and OOM when writing to a number of different indexes.
- We use a significantly enhanced (adding among other features quantized spaces) and turbo-charged (including support for x86 and ARM SIMD) HNSWlib for approximate nearest-neighbor search, and efficient mmap-backed offset storage for text retrieval. The system supports sharded (multi-threaded) HNSW indices, multiple search modes (kNN, radius, relative, adaptive, epsilon), deletion/undelete, merges, and incremental on-disk flushing.
- It also includes training for hyperparameter optimization.

HNSWlib Hyperparameter Optimisation

Hyperparameter Tuner using run-time adaptive adjustments.

- EfSearchTuner: determine a better choice for ef.

◆ **Speed-critical (real-time)**	`16-64`	Fast, but may miss some near neighbors.
◆ **Balanced (default)**	`100-200`	Common sweet spot (used by FAISS, sentence-transformers).
◆ **High recall / offline batch**	`300-800`	High accuracy, slower query.
◆ **Max recall / evaluation**	`1000+`	Nearly exact results, but 10× slower.

- EpsilonTuner: determine a better choice for epsilon in epsilon search.

Situation	Behavior
Too few results (<80% of target)	Increase ϵ slightly (expand radius).
Too many results (>120% of target)	Shrink ϵ slightly (tighten radius).
Stable result density	ϵ converges.

Performance Improvements

Turbo-charging the already lightning fast HNSWlib with a large number of SIMD optimisations.

L2 Distance: 3–5x faster than scalar
Inner Product: 3–5x faster than scalar
Apple Silicon (M1/M2/M3): 4–6x faster
AWS Graviton: 3–4x faster

Performance Boost using 03 and Neon (NEON) over straight
compile (Scalar): 20x

Platform: Apple M1 Pro
clang version 20.1.7
Darwin Kernel Version 24.6.0

L2 Distance (768 dimensions, 100000 iterations):
Scalar: 2796.45 ns
NEON: 139.22 ns
Speedup: 20x

Inner Product (768 dimensions, 100000 iterations):
Scalar: 2636.55 ns
NEON: 132.53 ns
Speedup: 20x

The performance improvement with the new NEON code
is on the order of 4.5x (Inner Product) – 6x (L2)

L2	Throughput: 841 vectors/sec.	QPS: 424 queries/sec
Inner Product	Throughput: 869 vectors/sec	QPS: 459 queries/sec

SIMD Platforms

Our main focus for SIMD is Intel/AMD x86-64 platforms and ARM 64bit.

- SSE (Streaming SIMD Extensions using 128-bit registers and floating-point operations)
- AVX2 (Advanced Vector Extensions using 256-bit registers)
- AVX512 (Advanced Vector Extensions 512)
- ARM NEON (Standard on nearly all ARM processors. It is a mandatory feature for Android devices running version 6.0 or higher)
- ARM SVE (Scalable Vector Extensions).

While NEON is common SVE is currently available only with [Fujitsu A64FX](#), [AWS Graviton3](#) and [Graviton4](#), [Alibaba Yitian 710](#), [Google Axion](#), and the [ARM Cortex-A710](#) and [Neoverse V1](#) cores. In particular not on Apple M-silicon at this time.

Quantization

Quantization is used to dramatically reduce memory usage and storage requirements, making it more scalable, especially for large datasets.

This system provides alongside FLOAT32, Quantization to Binary (1-bit), Ternary (1.58 bit), Nibble (4-bit) and Octet (8-bit). It also supports (not really quantisation but packing) FP16 and BF16.

Quantisation is realised through a selection of algorithms: Standard, Better, Centroid, Rotational, RaBitQ and RabbitQ-Extended.

Many models are already pre-quantised so we also implemented a “pass-through” (PASS) to enable native support of 1,2,3,4,5,6,8 and 16-bit integer and Bf16 and Fp16 pre-quantised models. This is useful for **small models** like all-MiniLM-L6-v2 with 4-bit quantization (which is only 14MB).

- We also support optional re-scoring. It gives you speed of quantized search with accuracy close to full-precision!
- For binary we’ve found RaBitQ to be currently the best:
- PASS: Since many GGML/GGUF models are already quantised we allow for pass-through quantisation, e.g. the vectors are decoded and packed into their size. A 4-bit model, for instance remains internally as 4-bit.

Quantization Performance

★ Use BIN1-RABITQ for production

Dimension: 384 Data points: 50000 Queries: 1000
Params: M = 32, ef_construction = 400, ef_search = 400, k = 10

Mode	Build (ms)	Query (ms)	Recall	Bytes/Vec	Memory (KB)
BASELINE-L2 (FLOAT32)	23317		0.9864		
INT8-STANDARD	18228.4	363.46	0.9885	384	18750
INT8-CENTROID	18483.1	375.71	0.9864	384	18750
INT8-ROTATIONAL	19952.2	413.05	0.9872	384	18750
INT4-STANDARD	77728.5	1359.43	0.9903	192	9375
INT4-ROTATIONAL	81096.9	1368.87	0.9859	192	9375
BIN1-STANDARD	4884.9	83.97	0.8504	48	2343
BIN1-BETTER	3380.7	49.37	0.6087	48	2343
BIN1-CENTROID	4807.9	81.08	0.8205	48	2343
BIN1-ROTATIONAL	8913.5	134.93	0.8452	48	2343
BIN1-RABITQ	4324.5	76.55	0.9999	112	5468
BIN1-RABITQ-EXT	7041.7	167.17	0.9987	304	14843

=== Analysis ===
Best recall: BIN1-RABITQ (0.9999)
Fastest query: BIN1-BETTER (49.37 ms)
Smallest memory: BIN1-STANDARD (2343 KB)

Development Organisation

Under re-lsearch.org (Munich) in association with ExoDAO Network Association (Zurich)

- re-lsearch.org is a newly formed organisation to centre the development of the re-lsearch engine. It is based in Munich, Germany.
- ExoDAO.net is a Swiss Association established in 2022 in Zurich, Switzerland around the Student Project House (SPH) of the Swiss Federal Institute of Technology in Zurich (ETH-Z). It has been tasked with a “moonshot” to develop a next generation Internet search. Its founding was catalysed by a grant from the Open Data Switzerland/Mercator Foundation Switzerland. The team is currently divided between Zurich (CH), Cambridge USA (Harvard) and Munich.
- Through the founding of re-lsearch.org the development of re-lsearch is disentangled from ExoDAO protocol development. It is intended as a clear signal that one can use re-lsearch without participating in the ExoDAO Network.
- Project Schmate provides code that may also be used outside of re-lsearch. This also detangles it as it has many other uses as a vector store. Since its C++ it can easily be embedded into a number of other environments such as Python.

Main Repository

<https://github.com/re-lsearch/Schmate>



This project was funded through the [NGI0 Commons Fund](#), a fund established by [NLnet](#) with financial support from the European Commission's [Next Generation Internet](#) programme, under the aegis of [DG Communications Networks, Content and Technology](#) under grant agreement N° [101135429](#). Additional funding is made available by the [Swiss State Secretariat for Education, Research and Innovation \(SERI\)](#).